

UNIVERSIDAD DE EL SALVADOR
FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS
SISTEMAS EMBEBIDOS I - EBB 115 CICLO 01 2026



PROYECTO DE INVESTIGACIÓN AVANCE NO. 1. PROYECTO DE DOMÓTICA CON ARDUINO.

TEMA:

“DISEÑO DE UN SISTEMA DOMÓTICO Y DE SEGURIDAD DISTRIBUIDO BASADO EN ARDUINO, UTILIZANDO COMUNICACIÓN I2C Y MÁQUINA DE ESTADOS PARA CONTROL INTELIGENTE DEL HOGAR.”

TUTOR RESPONSABLE:

MG. DENIS ALFREDO ALTUVE SANTAMARIA.

INTEGRANTES:

CARDOZA DIAZ, OSWALDO ISAIAS - CD15015

FRANCO DE LÓPEZ, INGRID JACQUELINE - FL18007

VÁSQUEZ ALEGRÍA, KEVIN DANIEL - VA18045

VÁSQUEZ MENJÍVAR, LUIS ENRIQUE - VM15037

CASTANEDA ROQUE, MARIO JOSUE - CR23036

CIUDAD UNIVERSITARIA, 24 DE MAYO DEL 2026

CONTENIDO

RESUMEN EJECUTIVO	3
Descripción del proyecto.....	3
Objetivo principal.....	3
FUNCIONALIDADES MAS IMPORTANTES	3
Tecnologías utilizadas	3
INTRODUCCION	4
Planteamiento del problema	5
Enunciado de la propuesta.....	5
Requisitos y materiales.....	6
Principales componentes electrónicos y de software empleados	6
Lista de materiales:	7
Herramientas de diseño y simulación utilizadas	8
Diseño del sistema.....	8
Desarrollo del código	11
Organización del código.....	11
Funciones clave implementadas.....	13
Conclusiones y mejoras a futuro	14
ANEXOS.....	16
Referencias.....	17

RESUMEN EJECUTIVO

DESCRIPCIÓN DEL PROYECTO

Este proyecto consiste en el desarrollo de un sistema domótico y de seguridad residencial distribuido, el cual divide las tareas de procesamiento entre dos placas Arduino conectadas mediante el protocolo I2C. El propósito es separar la adquisición de datos del control principal para lograr un sistema modular de bajo costo.

OBJETIVO PRINCIPAL

Diseñar e implementar una solución accesible y eficiente para gestionar de forma centralizada la seguridad perimetral, la iluminación automatizada y la regulación térmica de una vivienda en tiempo real.

FUNCIONALIDADES MAS IMPORTANTES

El sistema cuenta con un control de acceso con contraseña mediante teclado y servomotor , un sistema de seguridad inteligente basado en una máquina de estados (con modos de armado y alerta) , control automático de ventilación según la temperatura , y una interfaz LCD para que el usuario visualice el estado general de la casa.

TECNOLOGÍAS UTILIZADAS

Se utilizan dos placas Arduino Uno R3 comunicadas por bus I2C. Entre los componentes clave se incluyen sensores de movimiento PIR, sensores magnéticos y de temperatura TMP36, además de actuadores como un teclado matricial 4x4, pantalla LCD 16x2, servomotor, motores de corriente continua, zumbador y luces piloto.

INTRODUCCION

Hoy en día, la automatización de las casas, o domótica, ya no es un lujo de películas, sino una necesidad real para mejorar la seguridad, el ahorro de energía y la comodidad en el hogar. Sin embargo, el gran problema actual es que los sistemas comerciales que controlan las alarmas, las luces y la temperatura suelen ser muy caros y difíciles de centralizar en una sola plataforma. Por esta razón, el uso de tecnologías accesibles como los microcontroladores de código abierto se ha vuelto una alternativa excelente y de bajo costo para desarrollar este tipo de proyectos.

Este proyecto de investigación surge precisamente para dar una solución práctica y económica a esa necesidad. Se propone el diseño y la implementación de un sistema domótico integral que divide el trabajo entre dos placas de desarrollo conectadas entre sí. Una de ellas se encarga exclusivamente de recibir los datos de diferentes sensores repartidos por la casa, como el movimiento, la temperatura y la apertura de puertas. La otra placa funciona como el cerebro central, tomando decisiones en tiempo real, manejando una pantalla y un teclado para el usuario, y activando las respuestas automáticas, que van desde encender la ventilación o las luces hasta bloquear cerraduras y activar alarmas en caso de una intrusión. Al final, lo que se busca es demostrar que con componentes sencillos se puede lograr un sistema seguro, inteligente y perfectamente escalable para cualquier vivienda.

PLANTEAMIENTO DEL PROBLEMA

En el contexto actual, la automatización de viviendas (domótica) busca mejorar la seguridad, eficiencia energética y confort de los usuarios mediante la integración de sensores, actuadores y sistemas de control. Sin embargo, muchos entornos residenciales carecen de soluciones accesibles que permitan gestionar de forma centralizada variables críticas como la seguridad perimetral, el control de iluminación y la regulación térmica.

El desarrollo de sistemas embebidos basados en microcontroladores como Arduino representa una alternativa viable para implementar soluciones de bajo costo. No obstante, uno de los retos principales consiste en diseñar una arquitectura eficiente que permita:

- La adquisición confiable de datos desde múltiples sensores.
- La toma de decisiones en tiempo real basada en dichos datos.
- La interacción con el usuario mediante interfaces físicas.
- La coordinación entre múltiples dispositivos mediante protocolos de comunicación.

En este sentido, surge la necesidad de diseñar un sistema domótico distribuido que utilice más de una unidad de procesamiento, separando responsabilidades entre adquisición de datos y control del sistema, garantizando modularidad, escalabilidad y eficiencia en el procesamiento.

ENUNCIADO DE LA PROPUESTA

Se propone el desarrollo de un sistema de domótica centralizado basado en dos placas Arduino interconectadas mediante el protocolo I2C, donde:

- Un nodo Esclavo se encarga de la adquisición y preprocesamiento de datos provenientes de sensores (movimiento PIR, sensor magnético de puerta y sensor de temperatura).
- Un nodo Maestro gestiona la lógica de control, la interacción con el usuario y la activación de actuadores (iluminación, alarma, ventilación, servomotor de cerradura y motor de cochera).

El sistema implementa múltiples funcionalidades automatizadas, entre ellas:

1. **Sistema de seguridad inteligente** basado en una máquina de estados (desarmado, armado total, armado nocturno, pre-alerta y alerta).
2. **Control de acceso mediante teclado matricial**, con validación de contraseña.

3. **Gestión de alarmas acústicas y visuales** ante detección de intrusión.
4. **Control automático de temperatura**, activando ventilación según umbrales definidos.
5. **Automatización de iluminación** como mecanismo disuasivo.
6. **Control de acceso físico** mediante servomotor (puerta) y motor (cochera).

La comunicación entre ambos nodos se realiza mediante el envío estructurado de datos empaquetados en una estructura de 4 bytes, optimizando el uso del bus I2C y garantizando integridad en la transmisión.

Además, el sistema incorpora una interfaz de usuario basada en LCD y teclado, permitiendo visualizar estados del sistema, temperatura, estado del clima y realizar operaciones de control.

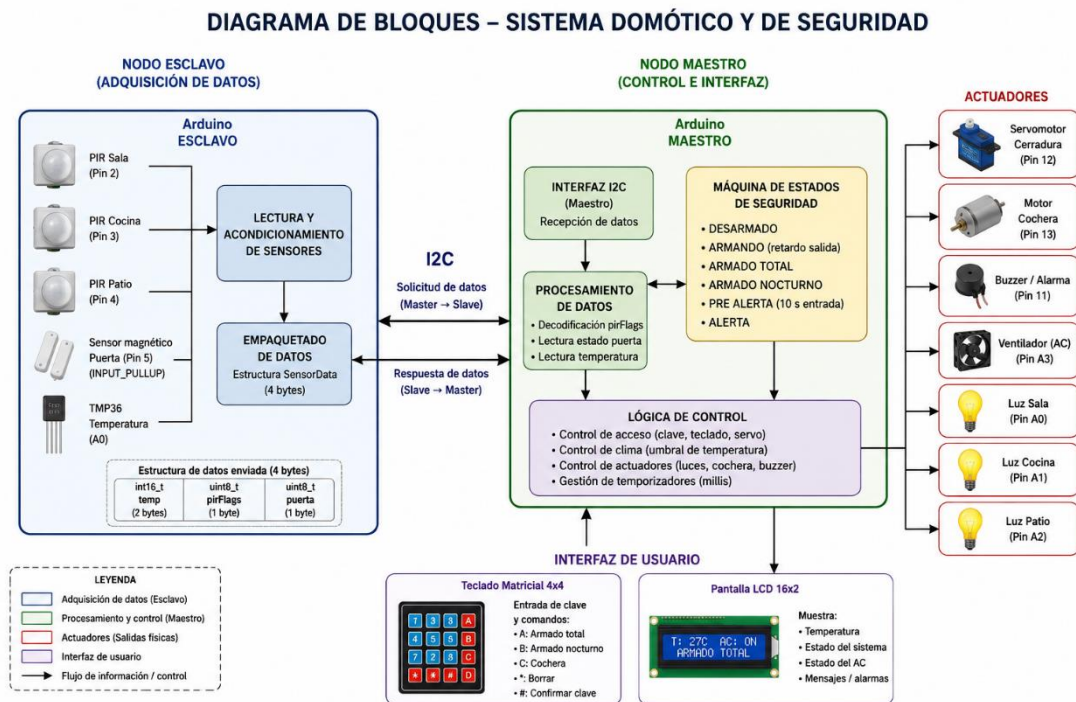


Figura 1. Diagrama de bloques como representación del funcionamiento del sistema. Creado en Figma.

REQUISITOS Y MATERIALES.

PRINCIPALES COMPONENTES ELECTRÓNICOS Y DE SOFTWARE EMPLEADOS

El sistema domótico desarrollado se basa en una arquitectura distribuida Maestro–Esclavo utilizando dos placas Arduino Uno R3 comunicadas mediante el protocolo I2C. Esta estructura

permite separar las funciones de adquisición de datos y control principal del sistema, mejorando la organización y modularidad del proyecto.

Entre los principales componentes electrónicos utilizados se encuentran sensores PIR encargados de detectar movimiento en diferentes áreas de la vivienda, un sensor magnético para supervisar la apertura y cierre de puertas, y un sensor de temperatura TMP36 utilizado para medir la temperatura ambiente y controlar automáticamente el sistema de ventilación.

Para la interacción con el usuario se implementó un teclado matricial 4x4 y una pantalla LCD 16x2 con interfaz I2C, permitiendo ingresar contraseñas, visualizar estados del sistema y controlar distintas funciones de seguridad y automatización.

El sistema también incorpora diversos actuadores, entre ellos un servomotor que simula una cerradura electrónica para el control de acceso, motores DC para la apertura de cochera y ventilación, un buzzer utilizado como alarma sonora y luces piloto que representan la iluminación automatizada de diferentes áreas de la vivienda.

Además, para la etapa de potencia se utilizaron transistores NPN y resistencias electrónicas, los cuales permiten activar motores y actuadores sin sobrecargar las salidas de los microcontroladores Arduino.

En el apartado de software, el proyecto fue desarrollado utilizando el entorno Arduino IDE y el lenguaje de programación C/C++. También se emplearon librerías específicas como Wire.h para la comunicación I2C, LiquidCrystal_I2C para el manejo de la pantalla LCD, Keypad para el teclado matricial y Servo.h para el control del servomotor.

LISTA DE MATERIALES:

Componente Principal	Función y Modelo Clave	Cantidad	Categoría
Arduino Uno R3	Microcontrolador de procesamiento principal de 8 bits	2 unidades	Procesamiento
Pantalla LCD 16x2 I2C	Visualizador de datos alfanuméricos en tiempo real	1 unidad	Interfaz de Usuario
Teclado Matricial 4x4	Ingreso manual de contraseñas de seguridad	1 unidad	Interfaz de Usuario
Sensores (PIR, TMP36, Magnético)	Sensores de movimiento, térmico analógico e intrusión	1 de cada uno	Entradas / Captación

Actuadores (Servo, Motores DC, Buzzer)	Cerradura simulada, ventilación, garaje y alarma sonora	1 de cada uno	Salidas / Físico
Etapas de Potencia (NPN, Resistencias)	Transistores 2N2222 y resistores de control de acoplamiento	Varios	Potencia y Seguridad

Tabla 1. Lista de componentes utilizados en simulador Tinkercad

HERRAMIENTAS DE DISEÑO Y SIMULACIÓN UTILIZADAS

Para el diseño y validación del sistema se utilizó la plataforma [Tinkercad](#), la cual permitió realizar la simulación completa del circuito electrónico, integrando sensores, actuadores, comunicación I2C y programación de ambas placas Arduino.

La programación y compilación del código se realizó mediante el entorno de desarrollo [Arduino IDE](#), utilizando programación estructurada y documentación interna del código para facilitar su comprensión y mantenimiento.

Las principales herramientas utilizadas en el proyecto fueron las siguientes:

Herramienta	Función
Tinkercad Circuits	Diseño y simulación del sistema domótico
Arduino IDE	Programación de microcontroladores Arduino
Librería Wire.h	Comunicación serial I2C entre nodos
Librería LiquidCrystal_I2C	Control de pantalla LCD
Librería Keypad	Manejo del teclado matricial
Librería Servo.h	Control del servomotor
Lenguaje C/C++	Desarrollo de la lógica del sistema

DISEÑO DEL SISTEMA

El sistema implementa una arquitectura distribuida Maestro–Esclavo mediante el protocolo I2C, separando la adquisición de datos y el control de actuadores para optimizar el rendimiento y organización del sistema.

Nodo Esclavo (Adquisición de Datos): El nodo esclavo se encarga de monitorear continuamente los sensores ambientales y perimetrales del sistema.

a. Entradas digitales

- Sensor PIR Sala → Pin 2
- Sensor PIR Cocina → Pin 3
- Sensor PIR Patio → Pin 4
- Sensor magnético de puerta → Pin 5

Entrada analógica: Sensor de temperatura TMP36 → Pin A0

Comunicación: El intercambio de datos con el nodo maestro se realiza mediante el bus I2C utilizando las líneas SDA y SCL dedicadas del Arduino.

Nodo Maestro (Control e Interfaz): El nodo maestro recibe y procesa los datos provenientes del esclavo, administra la lógica principal del sistema y controla los actuadores.

Interfaz de usuario

- Teclado matricial 4x4 para autenticación y control
- Pantalla LCD 16x2 con interfaz I2C

Actuadores

- Luz Sala → Pin A0
- Luz Cocina → Pin A1
- Luz Patio → Pin A2
- Ventilador → Pin A3
- Buzzer de alarma → Pin 11
- Servomotor de cerradura → Pin 12
- Motor DC de cochera → Pin 13

Etapas de potencia

El ventilador y el motor DC son activados mediante transistores NPN para evitar sobrecargar las salidas del Arduino debido al consumo de corriente de los motores.

Protocolo de Comunicación I2C

Con el objetivo de reducir tráfico en el bus I2C, el nodo esclavo empaqueta toda la información en una trama compacta de 4 bytes:

<i>Byte</i>	<i>Tipo</i>	<i>Información</i>
1 y 2	int16_t	Temperatura multiplicada por 100
3	uint8_t	Flags de movimiento PIR
4	uint8_t	Estado del sensor magnético

Distribución de Flags PIR

- Bit 0 → Sala
- Bit 1 → Cocina
- Bit 2 → Patio

Este esquema reduce el uso de memoria y permite una comunicación eficiente entre ambos nodos.

Máquina de Estados Finitos (FSM): El sistema opera mediante una Máquina de Estados Finitos implementada en el nodo maestro:

- DESARMADO:** El acceso permanece libre y las alarmas están deshabilitadas. Las funciones de monitoreo ambiental y control automático del ventilador continúan activas.
- ARMANDO:** Estado temporal activado mediante los comandos del teclado. El buzzer emite pitidos intermitentes durante el tiempo de salida antes de pasar al modo armado.
- ARMADO TOTAL:** Todos los sensores PIR y el sensor magnético son supervisados constantemente.
- ARMADO NOCTURNO:** Solo permanecen activos:
 - PIR Patio
 - Sensor magnético de puerta

Los sensores internos son ignorados para permitir movilidad dentro de la vivienda.

PRE-ALERTA: Se activa cuando la puerta es abierta mientras el sistema está armado. Inicia una cuenta regresiva de 10 segundos para permitir el ingreso de la contraseña.

ALERTA: Estado crítico activado por:

- Intrusión detectada
- Tres intentos fallidos de contraseña

Acciones ejecutadas

- Activación continua del buzzer

- Parpadeo de luces
- Cerradura mantenida en posición de bloqueo

DESARROLLO DEL CÓDIGO

El desarrollo del código del sistema domótico se organizó bajo una arquitectura distribuida utilizando dos placas Arduino Uno R3, conectadas mediante el protocolo de comunicación I2C. Esta estructura permite separar las funciones de adquisición de datos y control del sistema, facilitando una mejor organización del programa, una lectura más ordenada de los sensores y una gestión centralizada de los actuadores.

La primera placa Arduino funciona como **nodo esclavo**, encargado de la adquisición y preprocesamiento de datos provenientes de los sensores. En este nodo se conectaron tres sensores PIR para detectar movimiento en distintas áreas de la vivienda, un sensor magnético tipo reed switch para verificar el estado de la puerta principal y un sensor de temperatura TMP36 para medir la temperatura ambiental. La segunda placa Arduino funciona como **nodo maestro**, responsable de recibir los datos del esclavo, procesarlos, mostrar información al usuario por medio de una pantalla LCD 16x2 y activar los actuadores según las condiciones programadas.

La guía del proyecto solicita que el sistema utilice dos placas Arduino comunicadas mediante un protocolo serial como I2C o SPI, conectando sensores en una placa y actuadores e interfaz de usuario en la otra. Por ello, se eligió I2C, ya que permite una comunicación sencilla entre microcontroladores usando pocas líneas de conexión y facilita el intercambio de datos entre el nodo esclavo y el nodo maestro.

ORGANIZACIÓN DEL CÓDIGO

Código del Arduino esclavo: Este programa se encarga de leer los sensores físicos del sistema. En primer lugar, se importan las librerías necesarias, especialmente Wire.h, utilizada para habilitar la comunicación I2C. También se incluye stdint.h, que permite usar tipos de datos de tamaño fijo, como int16_t y uint8_t.

Luego se define una estructura llamada `SensorData`, la cual agrupa la información que será enviada al Arduino maestro. Esta estructura contiene tres campos principales:

- `temp`: almacena la temperatura leída por el sensor TMP36.
- `pirFlags`: guarda en un solo byte el estado de los tres sensores PIR.
- `puerta`: indica si la puerta principal se encuentra cerrada o abierta.

El uso de esta estructura permite enviar los datos de forma compacta y ordenada. En total, el paquete enviado tiene un tamaño de 4 bytes, lo cual optimiza el uso del bus I2C y evita enviar valores de manera desordenada o individual.

En la función `setup()`, el Arduino esclavo inicializa la comunicación I2C con la dirección 8 mediante la instrucción `Wire.begin(8)`. Además, se configura la función `enviar()` con `Wire.onRequest(enviar)`, de modo que cada vez que el Arduino maestro solicite información, el esclavo responda automáticamente enviando la estructura con los datos actualizados.

También se configuran los pines de entrada correspondientes a los sensores PIR y al sensor magnético de puerta. El sensor de puerta se configuró con `INPUT_PULLUP`, utilizando la resistencia interna del Arduino para evitar lecturas inestables.

En la función `loop()`, el programa actualiza constantemente el estado de los sensores. Primero limpia el valor de `pirFlags`, evitando que lecturas anteriores permanezcan almacenadas. Después, cada sensor PIR activa un bit específico dentro del byte de banderas. Por ejemplo, si el sensor de sala detecta movimiento, se activa el bit correspondiente usando una operación OR lógica. Este método permite representar varios sensores dentro de un solo dato.

Posteriormente, se lee el estado del sensor magnético de la puerta. Debido al uso de `INPUT_PULLUP`, una lectura en bajo representa que la puerta está cerrada. Por esa razón, el código convierte esa condición física en un valor lógico, donde 1 significa puerta cerrada y 0 significa puerta abierta.

Finalmente, el sensor TMP36 se lee desde el pin analógico A0. El valor obtenido del ADC se convierte primero a voltaje y luego a grados Celsius utilizando la fórmula correspondiente al sensor TMP36. El valor resultante se almacena en la variable `temp` dentro de la estructura de datos.

Código del Arduino maestro: El Arduino maestro se encarga del procesamiento central del sistema. Su función principal es solicitar información al nodo esclavo mediante I2C, interpretar los datos recibidos y ejecutar acciones según el estado del sistema. En este nodo se integran la pantalla LCD,

el teclado matricial, el servomotor de la puerta, el motor de cochera, el buzzer de alarma, el ventilador y las luces automatizadas.

El maestro utiliza una lógica basada en estados de seguridad. Los estados principales del sistema son:

- Desarmado.
- Armando.
- Armado total.
- Armado nocturno.
- Pre-alerta.
- Alerta.

Esta máquina de estados permite controlar el comportamiento del sistema de forma ordenada. Cuando el sistema está desarmado, el usuario puede abrir la puerta o cochera sin activar la alarma. Cuando está armado, el sistema monitorea los sensores y responde ante posibles intrusiones. En caso de detectar movimiento o apertura no autorizada de la puerta, el sistema puede pasar a pre-alerta y luego a alerta si no se ingresa la contraseña correcta.

FUNCIONES CLAVE IMPLEMENTADAS

Una de las funciones principales implementadas es el **sistema de seguridad inteligente**. Este sistema permite armar o desarmar la vivienda mediante el teclado matricial. El usuario puede ingresar una contraseña numérica y confirmarla con la tecla #. Si la contraseña coincide con la clave configurada, el sistema cambia de estado y permite realizar acciones como desarmar la alarma o abrir la puerta mediante el servomotor. Si la contraseña es incorrecta, se muestra un mensaje de error y se activa el buzzer como advertencia.

Otra función importante es el **armado total del sistema**, activado mediante la tecla A. En este modo, todos los sensores de movimiento y el sensor de puerta permanecen activos. Si alguno detecta actividad no autorizada, el sistema entra en estado de alerta. También se implementó el **armado nocturno**, activado con la tecla B, el cual permite proteger áreas externas o específicas sin afectar completamente el interior de la vivienda.

El sistema también incluye **control de acceso físico**. La puerta principal es controlada mediante un servomotor, el cual simula una cerradura electrónica. Cuando el sistema está desarmado y el usuario ingresa la contraseña correcta, el servo permite abrir la puerta. Además, se incluyó un motor para

simular la apertura y cierre de una cochera, controlado mediante la tecla C, siempre que el sistema no se encuentre en estado de alarma.

En cuanto al **control ambiental**, el sistema utiliza la lectura del sensor TMP36 para verificar la temperatura. Cuando la temperatura supera el umbral establecido, el Arduino maestro activa el ventilador, simulando un sistema de ventilación o aire acondicionado. Esto permite automatizar la regulación térmica del ambiente.

También se implementó una **automatización de iluminación**, utilizando tres focos que representan diferentes áreas de la vivienda, como sala, cocina y patio. Estas luces pueden activarse como respuesta ante detección de movimiento o como parte del sistema de seguridad, funcionando como mecanismo disuasivo ante una posible intrusión.

La **interfaz de usuario** se compone de una pantalla LCD 16x2 y un teclado matricial 4x4. La pantalla muestra información relevante como la temperatura, el estado del sistema, el estado del aire acondicionado o ventilación y mensajes de alarma. El teclado permite ingresar la contraseña, confirmar acciones, limpiar entradas y seleccionar modos de funcionamiento.

Comunicación entre nodos

La comunicación entre el Arduino maestro y el Arduino esclavo se realiza mediante el protocolo I2C. El esclavo permanece leyendo sensores y actualizando la estructura SensorData, mientras que el maestro solicita periódicamente esta información. Cuando el maestro realiza una solicitud, se ejecuta la función enviar() en el esclavo, enviando los 4 bytes correspondientes a temperatura, sensores PIR y estado de la puerta.

CONCLUSIONES Y MEJORAS A FUTURO

El desarrollo del sistema domótico permitió aplicar los principios fundamentales de los sistemas embebidos mediante el uso de microcontroladores Arduino, sensores, actuadores y comunicación entre dispositivos. A través de la simulación en Tinkercad, se logró diseñar una solución funcional orientada a la automatización de viviendas, integrando funciones de seguridad, control de acceso, iluminación y regulación de temperatura.

Uno de los principales resultados obtenidos fue la implementación de una arquitectura distribuida basada en dos placas Arduino. Esta división permitió separar la adquisición de datos del procesamiento principal, haciendo que el sistema fuera más ordenado, modular y fácil de comprender. El Arduino esclavo se encargó de leer sensores y enviar datos, mientras que el Arduino maestro concentró la lógica de control, la interfaz de usuario y la activación de los actuadores.

El uso del protocolo I2C resultó adecuado para la comunicación entre ambas placas, ya que permitió transmitir datos de manera compacta utilizando una estructura de 4 bytes. Esto demostró que es posible optimizar el intercambio de información entre microcontroladores y mantener una comunicación clara entre los módulos del sistema.

También se logró implementar un sistema de seguridad basado en estados, lo cual permitió organizar el comportamiento del sistema ante distintas condiciones. Los estados de desarmado, armado total, armado nocturno, pre-alerta y alerta permitieron simular un funcionamiento similar al de una alarma residencial real. Esta lógica hizo posible responder ante movimiento, apertura de puerta o ingreso incorrecto de contraseña.

La incorporación de una pantalla LCD y un teclado matricial permitió mejorar la interacción con el usuario. Gracias a estos elementos, el sistema no solo funciona de manera automática, sino que también permite visualizar información y ejecutar comandos manuales. Esto cumple con el requisito de contar con un medio de comunicación con el usuario, tal como lo solicita la guía del proyecto.

Como aprendizaje principal, se comprendió la importancia de estructurar correctamente el código en proyectos embebidos. Separar funciones, usar constantes para los pines, documentar el programa y organizar la información en estructuras facilita el mantenimiento del sistema y reduce errores durante la simulación. Además, se evidenció que una buena planificación del circuito y del flujo de datos es fundamental para lograr un funcionamiento estable.

Entre las mejoras futuras, se podría incorporar almacenamiento de eventos para registrar horarios de activación de la alarma, intentos fallidos de contraseña o cambios de estado del sistema. También sería posible agregar comunicación inalámbrica mediante módulos WiFi o Bluetooth, permitiendo controlar el sistema desde una aplicación móvil o una página web.

Otra mejora importante sería sustituir algunos actuadores simulados por módulos reales, como relés para controlar luces y ventiladores de corriente alterna, sensores de mayor precisión y cerraduras electrónicas reales. Además, se podría agregar una batería de respaldo para mantener activo el sistema durante cortes de energía.

En un escenario real, este sistema podría aplicarse en viviendas, oficinas pequeñas, locales comerciales o espacios que requieran monitoreo básico de seguridad y control automático de dispositivos. Su bajo costo y modularidad lo convierten en una alternativa viable para implementar soluciones domóticas accesibles, especialmente en ambientes donde se busca mejorar la seguridad, el confort y la eficiencia energética.

En conclusión, el proyecto cumplió con el objetivo de desarrollar un sistema de domótica centralizado basado en Arduino, integrando sensores, actuadores, comunicación I2C e interfaz de usuario. La simulación permitió validar el funcionamiento general del sistema y demostrar que los sistemas embebidos pueden ser utilizados para resolver necesidades reales de automatización en el hogar.

ANEXOS

LINK DEL CIRCUITO SIMULADO EN TINKERCAD:

- <https://www.tinkercad.com/things/0dHyecDn1ZZ-proyecto-etapa-1?sharecode=pZFCs7Gyt7GyALUTmS2kceD7lmsnv-2QzguE5xffuuk>

LINK DEL CODIGO IMPLEMENTADO EN EL REPOSITORIO

- https://github.com/LuisVasquezVM15037/EBB_ProyectoEtapa01
- https://github.com/LuisVasquezVM15037/EBB_ProyectoEtapa01.git

LINK DEL VIDEO DEL PROYECTO:

- https://drive.google.com/file/d/12eBmqllFWCBiDhno4h4XU4x9GC6Zv9_f/view?usp=drivesdk

REFERENCIAS

- LiquidCrystal Library Arduino
Arduino. (2025). *LiquidCrystal Library*. Recuperado de
[https://www.arduino.cc/reference/en/libraries/liquidcrystal/PIR Motion Sensor Tutorial
Arduino](https://www.arduino.cc/reference/en/libraries/liquidcrystal/PIR%20Motion%20Sensor%20Tutorial%20Arduino)
- Random Nerd Tutorials. (2025). *Arduino with PIR Motion Sensor*. Recuperado de
<https://randomnerdtutorials.com/arduino-with-pir-motion-sensor/>